Martin Gonzalez Rico

Lambdas y Streams

No.	Respuesta	Explicación:
1	c.[apple]	filter(s -> s.startsWith("a")) conserva solamente las cadenas que empiezan con la letra "a". De los elementos "apple", "banana", "cherry" y "date", solamente "apple" cumple esa condición. Después, collect(Collectors.toList()) guarda el resultado en otra lista, por eso se imprime [apple].
2	b.Andrea, Josias, Kevin, Laila	El método sorted() ordena los elementos. Como son objetos String, los ordena alfabéticamente. La lista original contiene Kevin, Andrea, Laila y Josias, pero al ordenar obtenemos [Andrea, Josias, Kevin, Laila].
3	a.9, 8, 3, 1	Collections.reverseOrder() proporciona un comparador que invierte el orden natural de los números. La lista original tiene 3, 1, 9, 8, pero al ordenarla de mayor a menor queda [9, 8, 3, 1]. Después, collect(Collectors.toList()) guarda los elementos ordenados en otra lista.
4	b.apple banana grapes pears	El comparador (a, b) -> a.compareTo(b) compara dos cadenas y las ordena alfabéticamente. Por eso el arreglo que originalmente contiene banana, apple, pears y grapes termina como apple, banana, grapes y pears. Después, el for-each recorre el arreglo ordenado e imprime cada palabra con un espacio delante. No se produce ninguna excepción porque todos los elementos contienen cadenas válidas.
5	c.3	Primero, map(n -> n * 2) multiplica cada número por dos, transformando la lista [1, 2, 3, 4, 5] en los valores [2, 4, 6, 8, 10] dentro del flujo. Después, filter(n -> n > 5) conserva los números mayores que cinco: 6, 8 y 10. Por ultimo, count() cuenta esos tres elementos y devuelve 3.
6	e.[Mark]	La condición p -> !p.name.startsWith("J") selecciona los nombres que no empiezan con "J". Como John y Jane comienzan con esa letra, se eliminan del resultado y solamente queda Mark. El método toString() de la clase Person devuelve el nombre, por eso al imprimir la lista aparece [Mark].
7	e.true 10 FUNCTIONAL Hello, world!	Predicate<String> comprueba si "functional" tiene más de cinco caracteres y devuelve true; Function<String, Integer> utiliza String::length para devolver la longitud del texto, que es 10; Consumer<String> convierte la palabra a mayúsculas y la imprime como FUNCTIONAL; y Supplier<String> devuelve "Hello, world!". Cada resultado se imprime en una línea diferente.

8	d.false	Primero, el predicado original comprueba si f.i > 0. Después, p.or(f -> f.i > 0) combina esa misma condición con otra condición idéntica usando or(), así que sigue comprobando si el número es mayor que cero. Luego, and(f -> f.i < 0) exige también que el número sea menor que cero. Para new Foo(1), la primera condición es verdadera, pero la segunda es falsa; como ambas deben cumplirse por el and(), el resultado final es false
9	a.2	Primero, filter(p -> p.name.startsWith("J")) conserva John, Jane y Jane, porque esos nombres empiezan con "J". Después, distinct() elimina los elementos duplicados según equals() y hashCode(). Como la clase Person sobrescribe ambos métodos para comparar por nombre, las dos personas llamadas Jane se consideran iguales y solamente queda una. Por eso se cuentan dos personas distintas: John y Jane.
10	e.Compilation fails on line 3.	El problema está en la línea 3, donde se declara IFun g = (i1, i2) -> i1 * i2;. Antes de esa línea ya existen las variables locales i1 e i2, declaradas en la línea 2. Java no permite que los parámetros de esa lambda utilicen los mismos nombres que variables locales que ya están dentro del mismo ámbito. La primera lambda de la línea 1 sí es válida porque en ese momento todavía no se han declarado esas variables locales. Por eso el código no compila y nunca llega a imprimir el resultado de la multiplicación.